

Systematic mapping of support tools for microservices

Jaime Sayago-Heredia¹, Yusleidy Godoy¹, Xavier Quiñonez-Ku¹, Homero Velastegui¹ and Gustavo Chango²

¹Pontificia Universidad Católica del Ecuador, Sede Esmeraldas, Ecuador
Espejo y subida a Santa Cruz Casilla 08-01-0065
jaime.sayago@pucese.edu.ec, yusleidy.godoy@pucese.edu.ec, xavier.quinonez@pucese.edu.ec, homero.j.velastegui.i@pucese.edu.ec

²Facultad de informática y electrónica, Escuela de Ingeniería en tecnologías de la información, ESPOCH, sede Orellana, Barrio Paraíso Amazónico. Calles Fray Gaspar de Carvajal, entre Quito y Napo.
wilson.chango@epoch.edu.ec

Abstract. The microservices architecture has gained popularity in software development for its scalability, flexibility, and independent deployment of services. However, there are significant technical challenges in its implementation, particularly concerning the support tools required for efficient operation and maintenance. Previous research has explored the state of microservices architecture, but a comprehensive and categorized analysis of specific support tools for microservices, including those for monitoring, security, automation, and deployment, is still lacking. This study aims to provide a comprehensive view of the current state of support tools for microservices, identifying key technologies and analyzing their applications and limitations. To achieve this goal, a Systematic Mapping Study (SMS) was conducted following the procedures of Petersen and Kitchenham, where 96 primary articles were collected and analyzed. The results include a taxonomy of tools organized into categories such as design and visualization, testing, monitoring, automation, and security, showing the diversity of approaches and their usefulness in specific environments. The research identifies critical areas where support tools require further development, such as the integration of advanced observability and automation tools. This analysis is useful for researchers and practitioners seeking to implement or improve microservices architectures in cloud applications.

Keywords: Microservices, Support tools, Programming languages, Taxonomy, Systematic mapping study.

1 Introduction

In the last decade, microservices architecture has emerged as one of the most influential trends in software development [1]. Its ability to divide monolithic applications into small, independent services [2], which can be autonomously developed deployed and scaled [3], has revolutionized the way modern applications are designed, deployed, and maintained [4]. This flexibility has allowed organizations to respond quickly to market changes [5] and user demands, improving scalability, resiliency, and the ability to

continuously innovation [6]. However, despite these numerous advantages, there is a notable gap in research specifically focused on the essential support tools for the effective implementation [7] and maintenance [8] of microservices architectures. The complexity inherent in managing multiple independent services requires a robust set of tools that facilitate aspects such as development [9], integration, deployment automation [10], monitoring [11] and security [9]. The current diversity of these tools makes it challenging for researchers and practitioners to navigate the microservices support landscape and select appropriate solutions. The objective of this study is to provide a comprehensive overview of the state of the art in support tools for microservices, focusing on the main challenges involved in implementing this architecture. Using an SMS [9] that follows the formal procedures defined by Petersen [12, 13] and Kitchenham [14], 69 primary studies were identified and analyzed, highlighting support tools for microservices. The goal is to provide an overview of the state of the art of research on support tools for microservices by focusing on the main challenges associated with microservices implementation [15]. To achieve this goal, a series of specific research questions was formulated, allowing for the breakdown and analysis of different aspects related to the use of tools in microservices. These questions cover topics such as the main reasons for adopting microservices, the predominant programming languages, as well as the most widely used tools and types of research prevalent in the field. This approach helps researchers and practitioners understand the available options, their benefits, and limitations, and identify areas requiring further research and development. The rest of this paper is structured as follows: Section 2 details the methods and tools. Section 3 presents the results of the SMS and discusses the main results of the study and analyses them. Finally, Section 4 presents the conclusions of this study.

1.1 Background

Some works aim to provide the state of the art in the field of tools for the development of microservices in general. Paper [3] discusses the current state of microservices architecture by identifying, classifying, assessing the state of the art, and offers a useful classification framework and provides a detailed assessment that gives a clear and rigorous picture of the current landscape in microservices architecture. Similarly, in the study Waseem et al. [16] It systematically classifies the literature and analyzes demographics, classification, and microservices methods, benefiting researchers and practitioners. Paper [17] investigates the quality attributes (QA) of microservice-based systems, concluding that scalability remains an indispensable concern, and that other QAs such as maintainability need more attention. In addition, the review [18] provides guidance on threats in microservices and how to detect, mitigate, or prevent them, categorizing threats in MSA and security mechanisms. Finally, paper [19] presents the state of the art in microservices for Internet of Things (IoT) applications offering a list of factors to consider when implementing microservices architectures in IoT and Mobile. It can be observed that literature reviews and systematic studies are limited to comparisons within a specific field. Similarly, there are several approaches and areas of microservices architecture. None of the studies takes a detailed look at support tools. The focus of this study is the identification of 69 articles for SMS, along with an analysis of the predominant programming languages and types of research prevalent in microservices.

2 Methods and Tools

2.1 Systematic Mapping Study Methodology

SMS is a secondary study that aims to classify and analyze previous research thematically [12, 20]. The objective of this SMS is to discover and understand the support tools available for the development of microservices architectures. SMS was conducted following the formal procedures defined by Petersen [13] and Kitchenham [14] and these steps of the research method are presented in this section.

2.1.1 Research Questions

We prepared the main research question **RQ**. What is the state of the art of support tools for developing a microservices architecture? The main research question (RQ) is to discover and evaluate recently published studies on support tools for microservices in different digital libraries. We segment our main research question into more specific research questions.

Table 1. Research Questions

Research Questions	Motivation
RQ1: What are the main reasons to use microservices?	Knowing if microservices offer enough advantages and their implementation is the most appropriate when developing cloud-based applications.
RQ 2: What are the main programming languages in microservices development?	Determine which are the main programming languages used for microservices development
RQ3: What are the support tools that exist for microservices?	Identify the novel and most commonly used tools for support in a microservices architecture based on paper research
RQ4. What kind of research has proliferated in this field?	Establish the most frequent type of research in this area, e.g., solution proposal, applied research, research evaluation, etc., in order to know the maturity of the area and identify gaps.

2.1.2 Search strategies

In systematic mapping, an important step is to define the process of searching for primary education. These studies are identified by searching scientific bibliographies or browsing research from specific journals and conferences known in the area. In this systematic mapping, four digital scientific databases considered relevant to software engineering, as recommended by Kurhamm et al. [21] were searched for primary studies: Scopus, IEEE Xplore, ACM Digital Library, and Springer. The search strings used for information collection were as follows

- a) (Microservices) AND (tools) OR (platforms)
- b) (TITLE (*Web Microservices*) AND TITLE (*Microservices Architecture*) AND NOT TITLE (*Monolithic System*))
- c) (Microservices) AND (tools) OR (Software)

We searched each of the four academic databases using the defined search string. An important point is that a filter was applied considering only studies from 2018 to 2024, thereby meeting one of the exclusion criteria. This selected period (seven years) is a prudent timeframe to avoid selecting outdated, general, or extensive works [22, 23]

Additionally, it helps to avoid works that are trendy with very short periods [24]. The inclusion and exclusion practices and strategies for studies are valuable for systematic reviews by Petersen and Gencel [25]. The following inclusion and exclusion criteria are defined (see Table 2).

Table 2. Inclusion and exclusion criteria.

Identification	Criteria
IC1	Peer-reviewed articles
IC2	The study employs a few types of support tools for microservices
IC3	The study provides some idea or type of application that could be applied in a microservices architecture.
IC4	Articles that were published between 01/01/2018 and 31/02/2024
EC1	Paper is duplicated
EC2	Articles that are not in Spanish
EC3	The work is preliminary research that is expanded or deepened in a more recent article by the same authors that has already been included.
EC4	The focus of the article is not within the area of computer science.

2.2 Extraction Process

The execution of our SMS was carried out by five researchers, with a period of 9 months to complete. The systematic mapping study timeline began with protocol development and improvement, duplicate removal and removal. Next, the study was selected by analyzing the title, abstract and keywords. Another iteration applying the inclusion and exclusion criteria. Next, download all the primary studies selected in this step. The selection process is determined by the full text, applying taxonomy and classification. Conflict resolution through focus group sessions. And finally, report on all the steps taken and the activities carried out throughout the study. Figure 1 show a summary of the search and selection process for primary studies and their respective results.

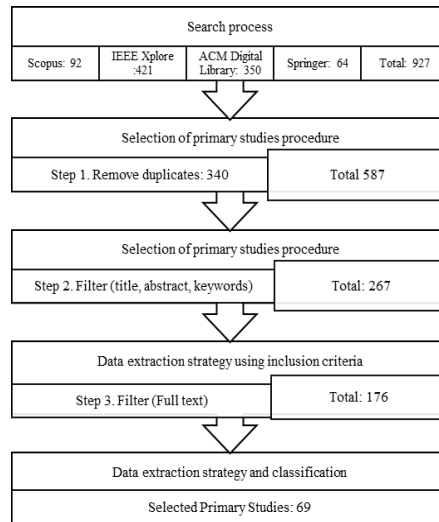


Figure 1. Results obtained from the search and selection process.

In total, 927 publications were obtained as a result of the automatic search in digital libraries, as a first step duplicates were eliminated (340 studies) obtaining a total of 587 studies. Then, applying the exclusion and inclusion criteria, 176 publications were selected. As the last one that corresponds to the complete reading of the study, 69 studies were selected. Data extraction, classification and synthesis were then carried out with these 69 primary studies. It is possible to visualize the tables obtained from the extraction, classification and synthesis of data online in <https://zenodo.org/records/12774068>.

3 Results and Discussions

3.1 RQ1: What are the main reasons to use microservices?

Throughout the development of our research, the different advantages of the implementation of microservices have been understood and analyzed. Microservices work independently of each other and remain loosely connected [26] along with better scalability [1] and facilitating better resource utilization [11]. In addition, it offers flexibility in terms of technologies and programming languages, allowing organizations to choose the best tools for each individual service, thus optimizing performance and productivity [27]. Ease of maintenance improves development speed and responsiveness to bugs or new functionality [16]. The standalone deployment capability allows applications to be updated, deployed, and scaled without needing to stop or affect other parts of the system [28], which is crucial for applications that require high availability [29]. Microservices integrate well with DevOps practices, improving deployment automation and continuous application management leads [30] to being applied in continuous development and testing [31]. While several advantages can be highlighted, microservices architectures present various challenges that complicate their implementation and operation. The choice of observability tools varies depending on the needs of the project, which can make it difficult to implement a cohesive observability strategy [11]. Furthermore, the lack of proper security testing and insufficient automatic scanning tools exacerbate vulnerabilities in microservices [9]. Microservices architectures are also targeted for targeted cyberattacks, introducing new challenges to their monitoring and defense [30]. Complexity in detecting and troubleshooting performance issues is another significant disadvantage, given that most current tools do not systematically cover the entire data flow [10]. The lack of support in integrated development environments (IDEs) for communication between services also adds complexity to the coding process. Finally, automated tools for compliance analysis, such as CATMA, are essential for detecting discrepancies in deployments, which shows the complexity inherent in maintenance [8]. Our analysis indicates that they offer numerous significant advantages for software development. These include improved scalability, flexibility, ease of maintenance, and independent deployment capability, allowing organizations to quickly adapt to market demands and improve operational efficiency. However, significant challenges are also identified, such as complexity in managing communication between services, security, and performance monitoring. Despite these challenges, the superiority of the

advantages evidence that microservices architecture is a robust and versatile option to address today's software development requirements

3.2 RQ 2: What are the main programming languages in microservices development?

In order to characterize the programming languages used in microservice development. It was discovered that various programming languages are predominantly used, as shown in Figure 2:

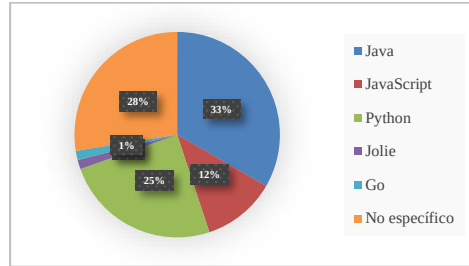


Figure 2. Result of programming languages most commonly used in microservices development.

Figure 2 describes Java (33%), Python (25%), JavaScript (12%), 28% of which is not specified, Jolie (1%) and Go (1%) as the most used language in microservices. This information reflects the flexibility in programming languages for the development of microservices tools. The use of these programming languages fits well with Viggiano's research through interviews he obtains similar information in terms of the most used programming languages, highlighting Java, JavaScript, C# and other technologies [32]. Similarly, a comparison was made with the consulting firm TIOBE, a company specialized in assessing and monitoring software quality [33]. This annual survey presents the most used programming languages in June 2024 compared to June 2023. Topping the list is Python with a vote of 15.39%, which has been awarded as the language of the year for the last five years according to the TIOBE Index [40]. In second place is C++ with a vote of 10.03%, and the C language is in third place with a vote of 9.23%. Java is in fourth position with 8.40%. When examining the most commonly used programming languages in microservices development, a remarkable diversity is observed. According to our analysis, Java, Python, and JavaScript stand out as the preferred languages. The diversity in the choice of programming languages demonstrates that there is no one-size-fits-all solution and that developers must consider multiple factors, including the nature of the project [34], the availability of resources [3], and the team's experience [2], to choose the most suitable language for their microservices needs.

3.3 RQ3: What are the support tools that exist for microservices?

Table 3 shows category classification performed for the research question. The selected studies exhibit different tools that require some kind of clustering. To accomplish

this, a taxonomy for microservices was created. This taxonomy is based on the taxonomy provided in [15] as well as other similar studies [35].

Table 3. **Taxonomy of support tools for microservices based on [15].**

Taxonomy				Studies	# Studies
Microservices architecture	Cross-cutting themes	Support Tools	Literature review	7,8,9,11,13,15,16,20,30,32,33,36,37,49, 50	15
			Design and visualization	2, 26,27,46,48, 52,56,65, 17	9
			Deployment and Upgrade	3,4, 10, 51,57, 61,64,66	8
			Testing	21,28,31, 38, 45, 62, 67	7
			Observability, Monitoring, and Metrics	19, 39,42, 47,63,69, 35, 41, 60	6
			Automation	20,23,24, 58, 68, 25	6
			Migration and extraction	5,6,14, 29, 55	5
			Development	40, 34,43,53, 12	5
			Safety	1,44, 59	3
			Benchmarking	18, 54	2

Table 3 shows how the selected primary studies are distributed in relation to (RQ3), the number of studies for each category. From this analysis, a collection of literature on microservices was refined to 69 relevant works, classified into: primary studies, i.e., literature that specifically investigates the research question RQ3 (using microservices, proposing frameworks and tools supporting microservices); and secondary studies, i.e., literature that reviews primary studies, reviews, and comparative studies that evaluate microservices or microservices-based approaches. Table 3 presents the taxonomy of support tools based on the classification of [15] as there is a wide (and growing) variety of approaches, this study was included in Garriga's subclassification and oriented toward the research question. The proposed distribution encompasses different support tools for building various characteristics of a microservices architecture. It can be stated that the different systematic literature reviews on microservices indicate that this architectural style is consolidating as a preferred solution for scalable and rapidly evolving cloud applications [3, 28, 36, 37]. Systematic review conducted by Vural identifies current trends, motivations behind microservices research, emerging standards, and potential research gaps [36]. Innovative proposals emerge in the form of new approaches and tools, such as those related to design, automation, patterns, and security that facilitate the implementation and improvement of microservices-based systems. The integration of MSA with DevOps stands out in paper 13, which categorizes research in microservices development and operations in DevOps, tool support for MSA, and migration experiences to MSA [16]. However, some studies reveal challenges in designing, developing, and maintaining microservices-based systems, such as the lack of tool support and knowledge of systematic methods [38]. Security in microservices is another critical aspect addressed in paper 49, which classifies threats and proposes security mechanisms to mitigate them [18]. Testing in microservices, crucial due to its distributed and modular nature, faces challenges in integration, consistency, and security [39, 40].

Studies like 21 and 45 compare open-source testing tools and their application [41, 42] and Bunk8s highlights test integration in Kubernetes [43]. In the Observability category, the focus is on understanding the state and behavior of distributed systems from external signals [11]. Studies highlight the importance of monitoring and real-time metrics collection [6, 11, 44–46]. In Automation, methodologies and tools are presented to optimize and manage microservices architectures, such as RM2MS and HydraGen [47, 48] and the advanced intelligent auto-scaling system [49]. Migration and extraction are crucial in the evolution towards microservices architectures, with frameworks and methods to overcome migration challenges [5, 50], and tools for the automatic extraction of microservices [10, 51]. In development, systems such as a chatbot to facilitate maintenance and rapid prototyping tools are highlighted [52, 53]. Finally, in security, article 1 explores the detection of malicious behaviors with machine learning [54], paper 44 presents Pomegranate, a security tool suite [30] and study 59 proposes machine learning models to identify security discussions in microservices [55]. Support tools for microservices encompass a wide range of functionalities, from design and automation to testing, observability, and security. The diversity of available tools reflects the complexity and continuous evolution of microservices-based systems. As the use of microservices continues to grow, the demand for more specialized and efficient tools also increases. The integration of new solutions and innovative approaches in areas such as design, automation, and security is essential to address emerging challenges and improve the efficiency of microservices systems. However, it is crucial for researchers and professionals to stay updated with the latest trends and advancements to select and use the most suitable tools according to their specific needs. The continuous evolution of support tools not only facilitates the implementation and management of microservices but also contributes to the consolidation of this architectural style as a preferred solution for scalable and rapidly evolving cloud applications.

3.4 RQ4. What kind of research has proliferated in this field?

Figure 3 describes the arrangement of primary studies according to the research questions, which are analyzed below:

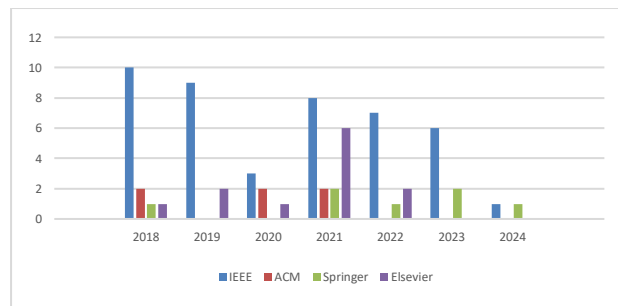


Figure 3. Distribution of studies per year.

Figure 3 indicates that the largest number of items extracted was in 2021 with 18, which means 26%. As for the second year with the highest amount, it was in 2018 with 14 articles, which corresponds to 20%. Continuing with the year 2019 with 11 articles that

correspond to 16% and the year 2022 10 articles were extracted that mean 14%, In 2020 it corresponds to 9% with 6 papers along with 2023 with 12% of a total of 8 articles. Finally, from 2024 only two articles were extracted, which correspond to 3%

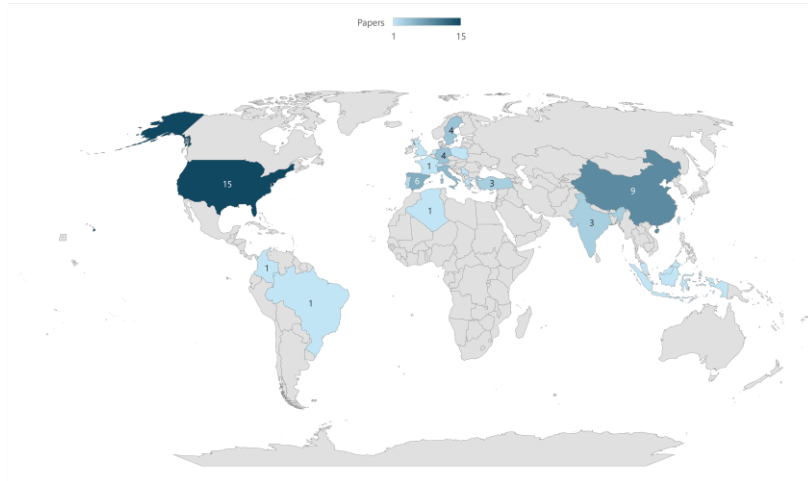


Figure 4. Number of items selected by country.

Figure 4 reveals that the selected studies are strongly concentrated in the United States and China, which represent 22% and 13.24% of the total of 69 articles, respectively. This suggests greater production and/or selection of research in these countries. However, Spain, Italy, Sweden, and Germany also have significant contributions, with percentages between 5.88% and 8.82%. Despite this concentration, there is a remarkable diversity with 25 countries represented, underlining the global interest and contribution in the area of study. Countries such as Taiwan, Portugal, Turkey and India, although with lower percentages, demonstrate active participation, highlighting the international nature of collaboration and research in microservices. This diversity suggests that microservices re-search is a global endeavor that benefits from a wide range of perspectives and knowledge. As for the types of documents, they were classified into two categories: lectures and articles. Papers were the most selected document type, with 43 papers (62%), followed by conference papers, with 26 papers (38%). Our analysis shows that studies in relation to microservices are in a trend of maturation to the formal that is reflected in the number of articles published, although it is also noteworthy that referring to conferences where partial advances or innovation are shown and that tend to be published in a shorter period of time.

3.5 Threats to validity

This section discusses the limitations of the systematic mapping study, based on the types of validity proposed by Petersen [13], as described below: In the context of the SMS, for descriptive validity, a review protocol was applied to ensure quality, validated by the authors. This protocol included verifying results through a random sample and using a validated data extraction form, as well as highlighting and marking relevant

texts to facilitate the review [56]. For theoretical validity, the risk of overlooking relevant factors by using a defined synonym term in the search was addressed, which was validated in the pilot search [56]. The generalizability of the results was affected by limitations such as publication bias, which was mitigated by using five relevant scientific databases [13]. External generalizability was also at risk due to possible subjectivity in the selection and classification of studies, although author review cycles were applied to minimize these risks [57]. Finally, interpretative validity was achieved when the conclusions were reasonable and based on the obtained data, with discussion and classification sessions to mitigate author bias in data interpretation [13].

4 Conclusions

In this work, an SMS of the research published in the last seven years was conducted, extracting and analyzing data from over 920 articles from four digital libraries, from which 69 relevant documents were selected. The systematic analysis of these articles focused on answering key questions about microservices architectures, providing a taxonomy that facilitates the understanding of support tools and the analysis of results. The research (RQ1) highlights the numerous advantages of microservices architectures, such as scalability, flexibility, ease of maintenance, and independent deployment capability, which allow organizations to adapt to market demands and improve operational efficiency. However, challenges in managing communication between services, security, and performance monitoring were also identified. The superiority of these advantages suggests that microservices are a robust option for software development (RQ1). Regarding the most used programming languages in microservices development, Java, Python, and JavaScript stand out for their flexibility and diverse capabilities, with Python being the rising leader due to its adaptability (RQ2). Additionally, a sub-taxonomy based on previous works (RQ3) was developed, grouping studies [15, 34, 35] according to key themes within microservices implementation and highlighting innovative approaches in security automation, monitoring, and deployment. Finally, the research reveals that microservices are preferred for scalable cloud applications, with a growing focus on design, automation, security, and observability (RQ4). This SMS provides a comprehensive and updated view of the state of the art in support tools for microservices, offering a valuable guide for researchers and professionals interested in this dynamic area of software development.

References

1. Soldani, J., Tamburri, D.A., Van Den Heuvel, W.J.: The pains and gains of microservices: A Systematic grey literature review. *Journal of Systems and Software*. 146, 215–232 (2018). <https://doi.org/10.1016/j.jss.2018.09.082>.
2. Santos, M., Neto, A., Matos, R., Araujo, J.: Software Aging and Rejuvenation in Microservices: A Systematic Literature Mapping. *Iberian Conference on Information Systems and Technologies, CISTI*. 2023, 2023 (2023). <https://doi.org/10.23919/CISTI58278.2023.10211574>.

3. Francesco, P. Di, Lago, P., Malavolta, I.: Architecting with microservices: A systematic mapping study. *Journal of Systems and Software*. 150, 77–97 (2019). <https://doi.org/10.1016/j.jss.2019.01.001>.
4. Hofmann, M., Schnabel, E., Stanley, K., Corporation., I.B.M., Organization., I.T.S.: *Ibm Microservices best practices for Java*. (2016).
5. Mazzara, M., Dragoni, N., Bucchiarone, A., Giaretta, A., Larsen, S.T., Dustdar, S.: Microservices: Migration of a Mission Critical System. *IEEE Trans Serv Comput*. 14, 1464–1477 (2021). <https://doi.org/10.1109/TSC.2018.2889087>.
6. Raj, V., Ravichandra, S.: Microservices: A perfect SOA based solution for enterprise applications compared to web services. In: 2018 3rd IEEE International Conference on Recent Trends in Electronics, Information and Communication Technology, RTEICT 2018 - Proceedings. pp. 1531–1536 (2018). <https://doi.org/10.1109/RTEICT42901.2018.9012140>.
7. Vera-Rivera, F.H., Puerto, E., Astudillo, H., Gaona, C.M.: Microservices Backlog—A Genetic Programming Technique for Identification and Evaluation of Microservices From User Stories. *IEEE Access*. 9, 117178–117203 (2021). <https://doi.org/10.1109/ACCESS.2021.3106342>.
8. Cao, C., Schneider, S., Díaz Ferreyra, N.E., Verwer, S., Panichella, A., Scandariato, R.: CATMA: Conformance Analysis Tool for Microservice Applications. *Proceedings - International Conference on Software Engineering*. 59–63 (2024). <https://doi.org/10.1145/3639478.3640022>.
9. Mateus-Coelho, N., Cruz-Cunha, M., Ferreira, L.G.: Security in microservices architectures. In: *Procedia Computer Science*. pp. 1225–1236. Elsevier B.V. (2021). <https://doi.org/10.1016/j.procs.2021.01.320>.
10. Mazlami, G., Cito, J., Leitner, P.: Extraction of Microservices from Monolithic Software Architectures. In: *Proceedings of the ACM Symposium on Applied Computing*. pp. 524–531 (2018). <https://doi.org/10.1109/ICWS.2017.61>.
11. Parmar, J., Sanghavi, S., Prasad, V., Shah, P.: Microservice Architecture Observability Tool Analysis. *Smart Innovation, Systems and Technologies*. 313, 1–8 (2023). https://doi.org/10.1007/978-981-19-8669-7_1.
12. Petersen, K., Feldt, R., Mujtaba, S., Mattsson, M.: Systematic Mapping Studies in Software Engineering. *Proceeding EASE’08 Proceedings of the 12th international conference on Evaluation and Assessment in Software Engineering*. Pages 68–77 (2008).
13. Petersen, K., Vakkalanka, S., Kuzniarz, L.: Guidelines for conducting systematic mapping studies in software engineering: An update. *Inf Softw Technol*. 64, 1–18 (2015). <https://doi.org/10.1016/j.infsof.2015.03.007>.
14. Kitchenham, B.A., Budgen, D., Pearl Brereton, O.: Using mapping studies as the basis for further research - A participant-observer case study. *Inf Softw Technol*. 53, 638–651 (2011). <https://doi.org/10.1016/j.infsof.2010.12.011>.
15. Garriga, M.: Towards a taxonomy of microservices architectures. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. 10729 LNCS, 203–218 (2018). https://doi.org/10.1007/978-3-319-74781-1_15.

16. Waseem, M., Liang, P., Shahin, M.: A Systematic Mapping Study on Microservices Architecture in DevOps. *Journal of Systems and Software*. 170, (2020). <https://doi.org/10.1016/j.jss.2020.110798>.
17. Li, S., Zhang, H., Jia, Z., Zhong, C., Zhang, C., Shan, Z., Shen, J., Babar, M.A.: Understanding and addressing quality attributes of microservices architecture: A Systematic literature review, (2021). <https://doi.org/10.1016/j.infsof.2020.106449>.
18. Hannousse, A., Yahiouche, S.: Securing Microservices and Microservice Architectures: A Systematic Mapping Study. no. pp. 0--21, (2020). <https://doi.org/10.1016/j.cosrev.2021.100415>.
19. Santana, C., Alencar, B., Prazeres, C.: Microservices: A mapping study for internet of things solutions. In: *NCA 2018 - 2018 IEEE 17th International Symposium on Network Computing and Applications* (2018). <https://doi.org/10.1109/NCA.2018.8548331>.
20. Kitchenham, B.: Guidelines for performing Systematic Literature Reviews in Software Engineering. (2007). <https://doi.org/10.1145/1134285.1134500>.
21. Kuhrmann, M., Fernández, D.M., Daneva, M.: On the pragmatic design of literature studies in software engineering: an experience-based guideline. *Empir Softw Eng*. 22, 2852–2891 (2017). <https://doi.org/10.1007/s10664-016-9492-y>.
22. Cosentino, V., Izquierdo, J.L.C., Cabot, J.: A Systematic Mapping Study of Software Development with GitHub. *IEEE Access*. 5, 7173–7192 (2017). <https://doi.org/10.1109/ACCESS.2017.2682323>.
23. Sayago-Heredia, J., Pérez-Castillo, R., Piattini, M.: A Systematic Mapping Study on Analysis of Code Repositories. *Informatica*. 32, 619–660 (2021). <https://doi.org/10.15388/21-INFOR454>.
24. De Farias, M.A.F., Colaço, M., Mendonça, M., Novais, R., Da Silva Carvalho, L.P., Spínola, R.O.: A systematic mapping study on mining software repositories. *Proceedings of the ACM Symposium on Applied Computing*. 04-08-April, 1472–1479 (2016). <https://doi.org/10.1145/2851613.2851786>.
25. Petersen, K., Gencel, C.: Worldviews, research methods, and their relationship to validity in empirical software engineering research. *Proceedings - Joint Conference of the 23rd International Workshop on Software Measurement and the 8th International Conference on Software Process and Product Measurement, IWSM-MENSURA 2013*. 81–89 (2013). <https://doi.org/10.1109/IWSM-Mensura.2013.22>.
26. Shabani, I., Mëziu, E., Berisha, B., Biba, T.: Design of Modern Distributed Systems based on Microservices Architecture. *International Journal of Advanced Computer Science and Applications*. 12, 153–159 (2021). <https://doi.org/10.14569/IJACSA.2021.0120220>.
27. Pathak, G., Singh, M.: A Review of Cloud Microservices Architecture for Modern Applications. In: *2023 World Conference on Communication and Computing, WCONF 2023* (2023). <https://doi.org/10.1109/WCONF58270.2023.10235199>.
28. A., N., Alshuqayran, R.E.N.: A systematic mapping study in microservice architecture. In: *Proceedings of the ACM Symposium on Applied Computing*. pp. 44–51 (2018). <https://doi.org/10.1109/SOCA.2016.15>.
29. Gupta, C., Van Ede, T., Continella, A.: HoneyKube: Designing and Deploying a Microservices-based Web Honeytrap. *Proceeding - 44th IEEE Symposium on Security and*

- Privacy Workshops, SPW 2023. 1–11 (2023). <https://doi.org/10.1109/SPW59333.2023.00005>.
30. Unver, B., Britto, R.: Automatic Detection of Security Deficiencies and Refactoring Advises for Microservices. In: Proceedings - 2023 IEEE/ACM International Conference on Software and System Processes, ICSSP 2023. pp. 25–34. Institute of Electrical and Electronics Engineers Inc. (2023). <https://doi.org/10.1109/ICSSP59042.2023.00013>.
 31. Wang, Y., Cheng, L., Sun, X.: Design and Research of Microservice Application Automation Testing Framework. In: Proceedings - 2019 International Conference on Information Technology and Computer Application, ITCA 2019. pp. 257–260 (2019). <https://doi.org/10.1109/ITCA49981.2019.00063>.
 32. Viggiano, M., Terra, R., Rocha, H., Tulio Valente, M., Figueiredo, E., Figueiredo Mi, E.: Microservices in Practice: A Survey Study. (2018).
 33. TIOBE: TIOBE, <https://www.tiobe.com/tiobe-index/>, last accessed 2024/06/26.
 34. Silva, F., Lelli, V., Santos, I., Andrade, R.: Towards a Fault Taxonomy for Microservices-Based Applications. In: Proceedings of the XXXVI Brazilian Symposium on Software Engineering. pp. 247–256. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3555228.3555245>.
 35. Weerasinghe, S., Perera, I.: Taxonomical Classification and Systematic Review on Microservices. *International Journal of Engineering Trends and Technology*. 70, 222–233 (2022). <https://doi.org/10.14445/22315381/IJETT-V70I3P225>.
 36. Vural, H., Koyuncu, M., Guney, S.: A systematic literature review on microservices. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. pp. 203–217. Springer Verlag (2017). https://doi.org/10.1007/978-3-319-62407-5_14.
 37. Soldani, J., Tamburri, D.A., Heuvel, W.J. Van Den: The pains and gains of microservices: A Systematic grey literature review. *Journal of Systems and Software*. 146, 215–232 (2018). <https://doi.org/10.1016/j.jss.2018.09.082>.
 38. Ghofrani, J., Lübke, D.: Challenges of microservices architecture: A survey on the state of the practice. In: *CEUR Workshop Proceedings*. pp. 1–8 (2018).
 39. Di Francesco, P., Lago, P., Malavolta, I.: Architecting with microservices: A systematic mapping study. *Journal of Systems and Software*. 150, 77–97 (2019). <https://doi.org/10.1016/j.jss.2019.01.001>.
 40. Sotomayor, J.P., Allala, S.C., Alt, P., Phillips, J., King, T.M., Clarke, P.J.: Comparison of runtime testing tools for microservices. In: *Proceedings - International Computer Software and Applications Conference*, vol. 2. pp. 356–361 (2019). <https://doi.org/10.1109/COMPSAC.2019.10232>.
 41. Hernández, C.M., Martínez, A., Quesada-López, C., Jenkins, M.: Comparison of End-to-End Testing Tools for Microservices: A Case Study BT - *Information Technology and Systems*. Presented at the (2021).
 42. Sotomayor, J.P., Allala, S.C., Alt, P., Phillips, J., King, T.M., Clarke, P.J.: Comparison of runtime testing tools for microservices. In: *Proceedings - International Computer Software and Applications Conference*, vol. 2, no. July. pp. 356–361 (2019). <https://doi.org/10.1109/COMPSAC.2019.10232>.
 43. Reile, C., Chadha, M., Hauner, V., Jindal, A., Hofmann, B., Gerndt, M.: Bunk8s: Enabling Easy Integration Testing of Microservices in Kubernetes. In: *Proceedings - 2022*

- IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2022 (2022). <https://doi.org/10.1109/SANER53432.2022.00062>.
44. Janes, A., Russo, B.: Automatic Performance Monitoring and Regression Testing During the Transition from Monolith to Microservices. In: 2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW). pp. 163–168 (2019). <https://doi.org/10.1109/ISSREW.2019.00067>.
 45. Yang, M., Huang, M.: An microservices-based openstack monitoring tool. In: Proceedings of the IEEE International Conference on Software Engineering and Service Sciences, ICSESS, vol. 2019-Octob. pp. 706–709 (2019). <https://doi.org/10.1109/ICSESS47205.2019.9040740>.
 46. Usman, M., Ferlin, S., Brunstrom, A., Taheri, J.: DESK: Distributed Observability Framework for Edge-Based Containerized Microservices. 2023 Joint European Conference on Networks and Communications and 6G Summit, EuCNC/6G Summit 2023. 617–622 (2023). <https://doi.org/10.1109/EuCNC/6GSummit58263.2023.10188344>.
 47. Li, Y., Zhang, Y., Yang, Y., Wang, W., Yin, Y.: RM2MS: A Tool for Automatic Identification of Microservices from Requirements Models. Proceedings - 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion, MODELS-C 2023. 50–54 (2023). <https://doi.org/10.1109/MODELS-C59198.2023.00018>.
 48. Sedghpour, M.R.S., Duque, A.O., Cai, X., Skubic, B., Elmroth, E., Klein, C., Tordsson, J.: HydraGen: A Microservice Benchmark Generator. IEEE International Conference on Cloud Computing, CLOUD. 2023, 189–200 (2023). <https://doi.org/10.1109/CLOUD60044.2023.00030>.
 49. Khaleq, A.A., Ra, I.: Intelligent Autoscaling of Microservices in the Cloud for Real-Time Applications. IEEE Access. 9, 35464–35476 (2021). <https://doi.org/10.1109/ACCESS.2021.3061890>.
 50. Salles, B., Cunha, J.: Visually-Assisted Decomposition of Monoliths to Microservices. In: Proceedings of IEEE Symposium on Visual Languages and Human-Centric Computing, VL/HCC (2023). <https://doi.org/10.1109/VL-HCC57772.2023.00057>.
 51. Akkaya, K., Ovatman, T.: A Comparative Study of Meta-Data-Based Microservice Extraction Tools. International Journal of Service Science, Management, Engineering, and Technology. 13, (2022). <https://doi.org/10.4018/IJSSMET.298677>.
 52. Huang, Y.-W., Ma, S.-P., Wang, S.K.: MsdoBot: An Extensible Chabot Platform for Microservice Development and Operations. In: Proceedings - 2022 IEEE International Conference on e-Business Engineering, ICEBE 2022. pp. 124–129 (2022). <https://doi.org/10.1109/ICEBE55470.2022.00030>.
 53. Stojkov, A., Stojanov, Z.: Review of methods for migrating software systems to microservices architecture. Journal of Engineering Management and Competitiveness. 11, 152–162 (2021). <https://doi.org/10.5937/jemc2102152s>.
 54. Kautsar, I.A., Maika, M.R., Budiman, A.N., Setyawan, A.B., Awali, J.Y.: Microservice Based Architecture: The Development of Rapid Prototyping Supportive Tools for Project Based Learning. In: EDUNINE 2023 - 7th IEEE World Engineering Education Conference: Reimagining Engineering - Toward the Next Generation of Engineering Education, Merging Technologies in a Connected World, Proceedings (2023). <https://doi.org/10.1109/EDUNINE57531.2023.10102884>.

55. Rezaei Nasab, A., Shahin, M., Liang, P., Basiri, M.E., Hoseyni Raviz, S.A., Khalajzadeh, H., Waseem, M., Naseri, A.: Automated identification of security discussions in microservices systems: Industrial surveys and experiments. *Journal of Systems and Software*. 181, (2021). <https://doi.org/10.1016/j.jss.2021.111046>.
56. Badampudi, D., Wohlin, C., Petersen, K.: Software component decision-making: In-house, OSS, COTS or outsourcing - A systematic literature review. *Journal of Systems and Software*. 121, 105–124 (2016). <https://doi.org/10.1016/j.jss.2016.07.027>.
57. Munir, H., Wnuk, K., Runeson, P.: Open innovation in software engineering: a systematic mapping study. *Empir Softw Eng*. 21, 684–723 (2016). <https://doi.org/10.1007/s10664-015-9380-x>.